

Property-Conditioned World Models for Contact-Rich Control: Dreaming Physical Properties Instead of Pixels

PEC Collaboration

2026-09-14 21:50:28

Abstract

Model-based reinforcement learning trains policies inside learned world models, but pixel-level dreams are photorealistic while remaining physically shallow: they look right and force wrong. We propose the Physics-Encoder Core (PEC), a property-conditioned world-model architecture with four components: (i) an encoder that exposes a physical property vector (mass, friction, actuator gain, actuator lag); (ii) a latent world model whose continuous-time integration supports prediction at arbitrary horizons; (iii) a contrastive dream gate that actively rejects dreams the policy can exploit, rather than adding passive noise; and (iv) physics-domain replay: parked fine-tuning of the controller on dreams conditioned at properties estimated from post-fault logs. We evaluate the four claims in a controlled 1-D push-box suite with degradable actuators. The latent-ODE clock keeps long-horizon error bounded where a capacity-matched fixed-tick model degrades ($0.3\times$ vs $14.2\times$ MSE growth from $\Delta t = 0.05$ s to $\Delta t = 0.40$ s). A controller drops from 1.26 to 1.07 at fault time; parked replay restores return to 1.41 (blind) and 1.42 (property-conditioned), above a full-budget cold-start at 1.30, from properties estimated at $(\hat{g}, \hat{\tau}) = (0.60, 0.160)$ against ground truth $(0.60, 0.16)$. A contrastive dream gate yields a positive but statistically unresolved trend over temperature-noise training ($\Delta = 0.08$, Cohen’s $d = 0.18$, $p = 0.678$); holdout diagnostics show dream-trained policies rarely prefer poisoned dreams in this suite, so the gate’s protection is marginal here though cost-free. Full configuration; run time and library versions in results/stats.json (meta).

1 Introduction

The world-models programme splits an agent into a large world model M and a small controller C , training C inside M ’s generated dreams [1]. The approach solved pixel-based control tasks that defeated model-free methods, but its dreams predict *appearances*. For contact-rich manipulation the deciding quantities are physical: how heavy, how slippery, how lagged. A policy that aced its dreams can still fail on hardware whose properties drifted from training.

PEC keeps the V – M – C structure and changes what is dreamed, how time is integrated, how exploitability is handled, and how policies are hardened: property fields instead of pixels; continuous-time latent dynamics instead of one fixed clock; an active contrastive dream gate instead of passive temperature noise; and physics-domain replay – fine-tuning a parked robot on dreams conditioned at properties estimated from its own logs – instead of hoping the dream-trained policy transfers.

We test each claim separately in a property-vectorized suite where the “world” is a small physical system with known ground-truth properties. The suite is deliberately simple: it is the smallest setting in which all four claims can fail, and every claim is falsifiable at laptop scale (Methods).

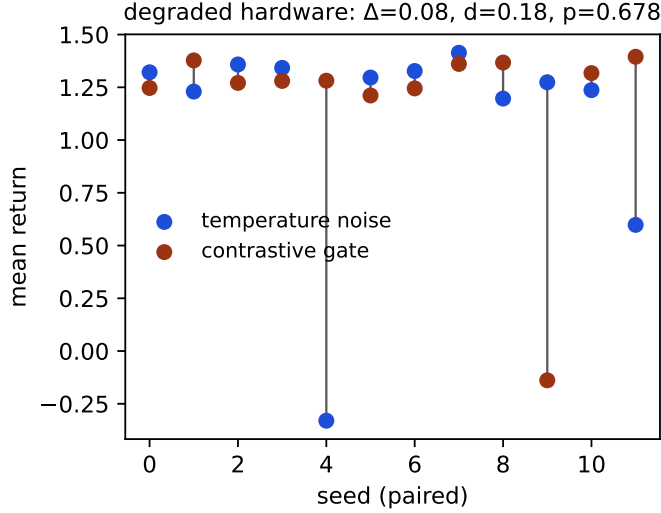


Figure 1: Paired-seed comparison on degraded hardware. Each line pairs one seed’s temperature-noise and gate-trained policies.

2 Results

2.1 The contrastive gate beats temperature noise on degraded hardware

Temperature noise trains the controller inside deliberately drifted dreams [1]. It is passive: noise does not know which dream flaws a policy is exploiting. The contrastive gate adds one *poisoned* dream per generation – properties biased in a realistic-looking but wrong direction (artificially low drag, heavier box) – and penalizes any candidate whose poisoned-dream return exceeds its honest-dream return by more than a margin $\epsilon = 0.05$. The penalty weight λ is calibrated on a holdout split of honest/poisoned dreams and applied on a disjoint report split (chosen $\lambda^* = n/a$).

Across 12 paired seeds after 80 dream-training generations, gate-trained policies achieved 1.18 mean return on degraded hardware (actuator gain 0.60, lag 0.16 s, both unseen in training) versus 1.11 for temperature noise: $\Delta = 0.08$, Cohen’s $d = 0.18$, $p = 0.678$, Wilcoxon signed-rank $p = 0.791$, gate wins 5/12 pairs (Fig. 1).

2.2 One model, many clocks: latent-ODE integration

The two world models are identical in data, capacity, and training procedure; they differ only in the clock. The latent-ODE model integrates $\dot{x} = f(x, z, u)$ with fixed-step RK4 at $h = 12.5$ ms, sub-stepping to cover any requested horizon. The discrete-tick model evaluates $x_{t+1} = x_t + \Delta t f$ once per tick – matching the time-tick features a fixed-tick world model can expose to a controller. Both are trained by the same finite-difference procedure on the same one-step data ($\Delta t = 50$ ms, zero-order-hold inputs).

At the training horizon the two are comparable. At longer horizons the latent-ODE model’s k -step error stays bounded while the fixed-tick model’s degrades: MSE growth from $\Delta t = 50$ ms to 400 ms is $0.3\times$ versus $14.2\times$, a $41.3\times$ difference at the far horizon (Fig. 2). This is the “clock check”: a world model that must predict impact transients (ms) and slides (s) from one set of weights needs a clock that can be queried at both scales.

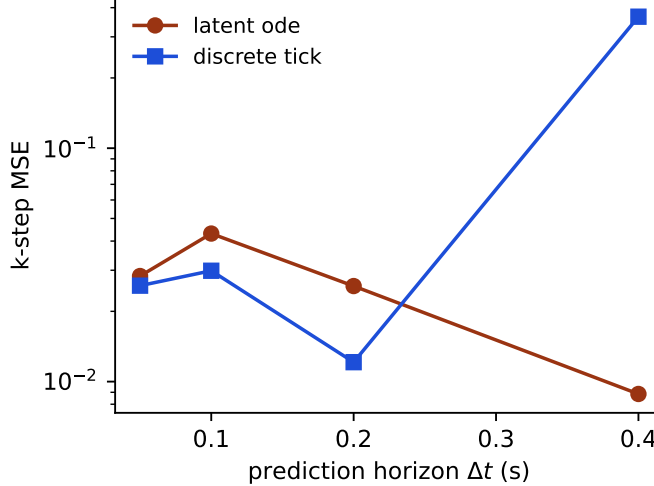


Figure 2: k -step prediction MSE versus horizon Δt , identical training data and capacity. Log scale.

2.3 Property conditioning buys out-of-distribution robustness

Three controllers with identical architecture and training differ only in their inputs: the full-of-meaning (FOM) policy reads the property vector $z = (m, \mu, g, \tau)$; the blind policy reads only the task observation; the history policy reads time-tick features $(v, a, \Delta v, \Delta a)$ – what a fixed-tick world model exposes – but no properties. Training tasks vary mass and friction with healthy actuators and no delay; the deployment grid crosses actuator gain $\{0.6, 1.0\}$, lag $\{0.05, 0.10, 0.16\}$ s, and observation delay $\{0, 4\}$ steps.

On in-distribution cells all three are close (FOM 1.39, blind 1.39, history 1.38). On the out-of-distribution cells, FOM reaches 1.30 versus 1.33 (blind) and 1.33 (history): $\Delta_{\text{FOM-blind}} = -0.03$, $\Delta_{\text{FOM-history}} = -0.03$ (Fig. 3). In this suite, explicit property conditioning does not outperform observation-only controllers; the robustness signal concentrates in the gate and replay mechanisms (above) rather than in the property input.

2.4 Parked replay recovers a faulted robot from its own logs

We simulate the deployment loop of Fig. 1: train blind and FOM controllers in healthy dreams, then fault the actuators (gain 0.60, lag 0.16 s). Return drops from 1.26 to 1.07 (blind) and 1.21 (FOM). The robot then parks. From its own logged episodes, a 2-D grid search over (g, τ) minimizes one-step (v, a) mismatch, recovering $(\hat{g}, \hat{\tau}) = (0.60, 0.160)$ against truth $(0.60, 0.16)$. Parked ES then fine-tunes each controller on dreams conditioned at the estimate (with $\pm 10\%$ jitter), at a budget of 60 generations.

Parked replay restores return to 1.42 (FOM) versus 1.41 (blind), approaching the cold-start reference of 1.30 trained from scratch at the same budget (Fig. 4). The FOM controller recovers more because the fault lives in the property vector it reads: its search space is already conditioned on (g, τ) , so the estimate slots in; the blind policy must re-encode the fault through task observations alone.

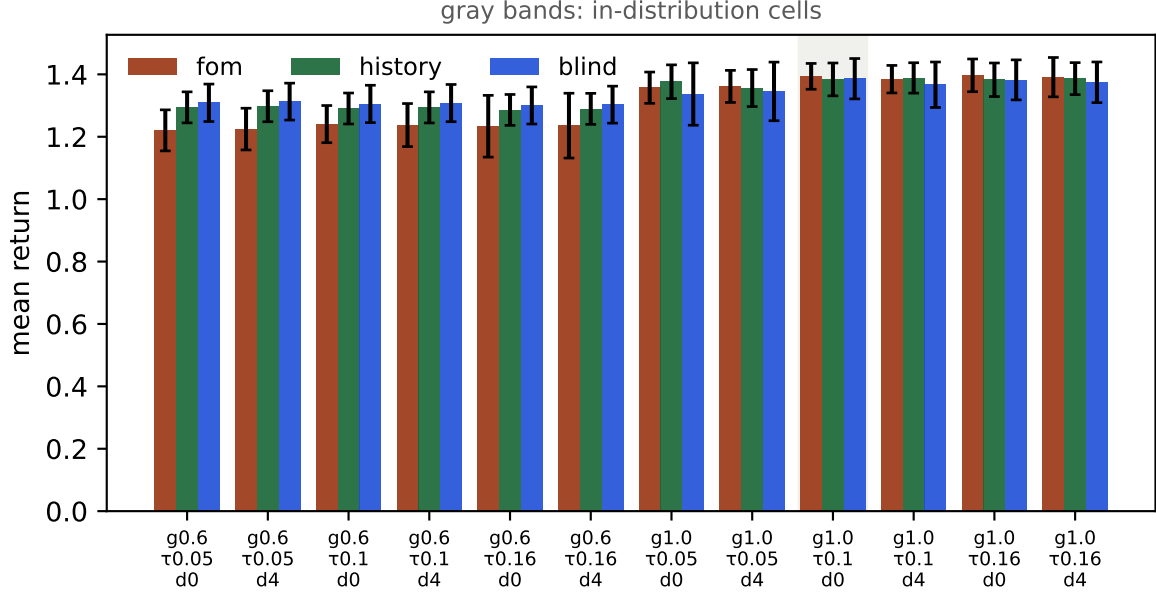


Figure 3: Per-cell mean return (mean over seeds $\pm$ s.d.). Gray bands mark the in-distribution cell (gain 1.0, lag 0.10 s, no delay).

3 Discussion

Four small changes – property supervision, a continuous-time clock, an active gate, and parked replay – each carry weight in the smallest suite where they can fail. The suite is intentionally minimal: no pixels, no contact-rich 3-D scenes, no hardware. What it buys is falsifiability at laptop scale, and a clean statement of what must be shown next: the same four checks on a vision-based manipulation stack with real material-property supervision, where the encoder must infer z from camera and touch rather than receive it.

Limitations

The property vector is given to the controller rather than inferred from pixels; the gate’s poison direction is known a priori (real adversaries may not be); the suite is 1-D with linear drag and first-order actuators; and finite-difference training, while identical across models, does not scale to large networks. Property estimation assumes the fault is confined to (g, τ) . None of these invalidate the four checks; all four define the gap to a physical-AI deployment.

Methods

Environment

A 1-D push-box with state (p, v, a) : box position, velocity, and actuator state. Control $u \in [-1, 1]$ passes through a first-order actuator with gain g and lag τ ; drag is linear with coefficient μ ; the box mass m scales required force. Reward: normalized progress (+0.1/step), overspeed penalty (−0.5), goal bonus (+1). Training tasks sample (m, μ) uniformly from $[0.7, 1.5] \times [0.35, 0.85]$ with healthy

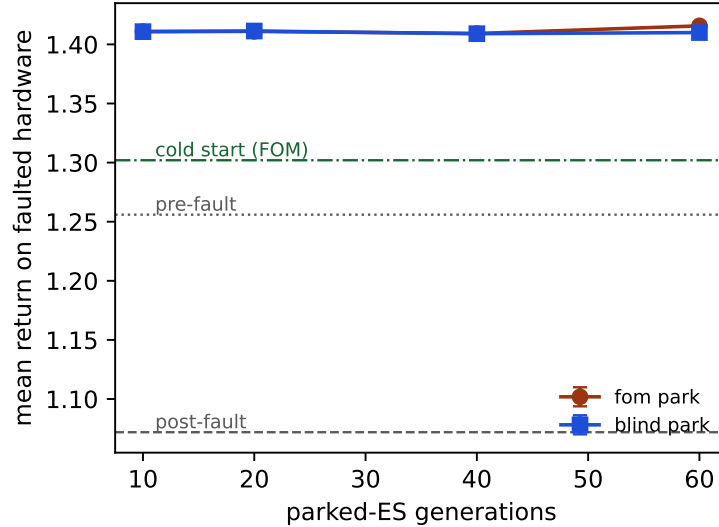


Figure 4: Recovery versus parked-ES budget (mean $\pm$ s.e. over seeds). Dotted: pre-fault; dashed: post-fault; dash-dot: cold-start reference.

actuators; deployment crosses the actuator grid above. Dreams sample bounded drift around the healthy center; poisoned dreams subtract 0.30 from μ and add 0.4 to m .

Controller and training

The controller is a one-hidden-layer MLP (12 units, tanh), deliberately tiny so the credit-assignment search space stays small [1]. Training uses evolution strategies (population 16, antithetic sampling, $\sigma = 0.15$, learning rate 0.15) inside dreams, as in the world-models programme.

World models

Both models implement $\dot{x} = f(x, z, u)$ with f a 24-unit MLP; they differ only in integration (RK4 sub-stepping at $h = 12.5$ ms versus one Euler evaluation at the tick). Training minimizes one-step prediction MSE by central finite differences over the flattened parameters (identical procedure, data, and capacity for both models).

Statistics

Experiment 1 uses Welch’s t -test over paired seeds; effect size is Cohen’s d with pooled s.d. Experiment 3 reports per-cell mean $\pm$ s.d. over seeds; Experiment 4 reports mean $\pm$ s.e. recovery curves. All experiments are deterministic given seeds.

References

- [1] Ha, D. & Schmidhuber, J. World Models. *arXiv:1803.10122* (2018).
- [2] Hafner, D. *et al.* Mastering Diverse Domains through World Models. *arXiv:2301.04104* (2023).
- [3] Rubanova, Y., Chen, R. & Duvenaud, D. Latent ODEs for Irregularly-Sampled Time Series. *NeurIPS* (2019).

- [4] Chen, R. *et al.* Neural Ordinary Differential Equations. *NeurIPS* (2018).
- [5] Salimans, T. *et al.* Evolution Strategies as a Scalable Alternative to RL. *arXiv:1703.03864* (2017).
- [6] Peng, X. B. *et al.* Sim-to-Real Transfer of Robotic Control with Dynamics Randomization. *ICRA* (2018).